

Laboratory 5

Variant D

Group 19

By Mathis Morra-Fischer & Nolan Capomaggio

Introduction

The objective of this assignment is to implement a Multilayer Perceptron from scratch using NumPy and to train it on the MNIST image classification dataset. The focus of Variant D is to compare the effect of different loss functions on the training and classification performance of the same neural network.

The three loss functions compared in this work are:

- Mean Squared Error,
- Mean Absolute Error,
- Cross-Entropy with softmax.

All experiments use the same activation function, ReLU, and the same optimiser, Stochastic Gradient Descent. The goal is to analyse how the choice of loss function affects convergence speed, stability, and final test accuracy.

Implementation

Mean Squared Error

The Mean Squared Error loss is defined as:

$$MSE = \frac{1}{n} \sum (y_p - y_t)^2$$

Its derivative with respect to the predictions is:

$$\frac{\partial MSE}{\partial y_p} = \frac{2}{n} (y_p - y_t)$$

MSE is commonly used for regression problems, but it can also be applied to classification by comparing the predicted output vector with the one-hot encoded target vector.

Mean Absolute Error

The Mean Absolute Error loss is defined as:

$$MAE = \frac{1}{n} \sum |y_p - y_t|$$

Its derivative is:

$$\frac{\partial MAE}{\partial y_p} = \frac{1}{n} \text{sign}(y_p - y_t)$$

MAE only uses the sign of the error in its gradient. This means that the gradient does not contain information about the magnitude of the error. As a result, training can be slower or less stable compared to MSE or Cross-Entropy.

Cross-Entropy with softmax

For Cross-Entropy, the output layer is linear. The network outputs raw scores called logits. Softmax is applied inside the loss function to convert logits into probabilities:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

The Cross-Entropy loss is:

$$CE = -\frac{1}{n} \sum y_t \log(\text{softmax}(y_p))$$

The combined derivative of softmax and Cross-Entropy is:

$$\frac{\partial CE}{\partial y_p} = \frac{1}{n} (\text{softmax}(y_p) - y_t)$$

This combined expression is numerically stable and avoids explicitly computing the full softmax derivative.

Cross-Entropy is expected to perform better for multi-class classification because it directly optimizes the probability assigned to the correct class.

Experimental setup

The experiments were performed on the MNIST dataset. Each image has size 28 \times 28, so each input is flattened into a vector of 784 values. Pixel values were normalized to the range [0, 1] by dividing by 255.

The labels were converted to one-hot encoded vectors of size 10.

Two architectures were tested:

Architecture 1: $784 \rightarrow 128 \rightarrow 10$

This architecture contains one hidden layer with 128 neurons and a final linear output layer with 10 outputs.

Architecture 2: $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$

This architecture contains two hidden layers with 256 and 128 neurons, followed by a final linear output layer.

For both architectures, ReLU was used after each hidden layer. No activation was applied after the final layer, because Cross-Entropy applies softmax internally.

The experiments were run with three random seeds: 0, 1, 2

The following hyperparameters were used:

Parameter	Value
Epochs	30
Learning rate	0.1
Optimiser	SGD
Activation	ReLU
Training subset size	10,000
Test subset size	2,000

For each loss function and architecture, the mean and standard deviation of the test accuracy were computed over the three seeds.

Results

Accuracy results

Architecture 784-128-10:

Loss function	Seed 0	Seed 1	Seed 2	Mean accuracy	Std
MSE	0.4330	0.5195	0.4490	0.4672	0.0376
MAE	0.2885	0.4090	0.2795	0.3257	0.0590
Cross-Entropy	0.8130	0.8085	0.8105	0.8107	0.0018

Architecture 784-256-128-10:

Loss function	Seed 0	Seed 1	Seed 2	Mean accuracy	Std
MSE	0.4755	0.4955	0.4605	0.4772	0.0143
MAE	0.3365	0.3180	0.3325	0.3290	0.0079
Cross-Entropy	0.8400	0.8455	0.8240	0.8365	0.0091

The best result was obtained with Cross-Entropy using the second architecture:
83.65% $\pm$ 0.91%

Training curves

The following figures show the training loss curves for each loss function.

Figure 1 — Training loss curves for architecture 784-128-10

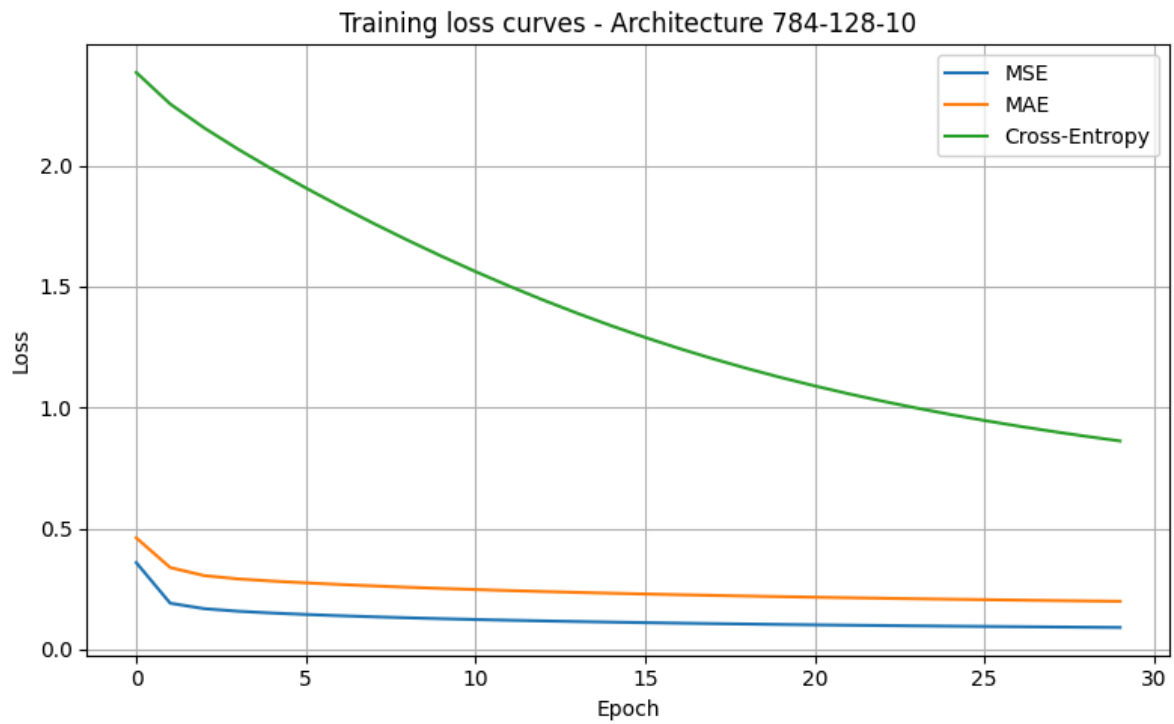
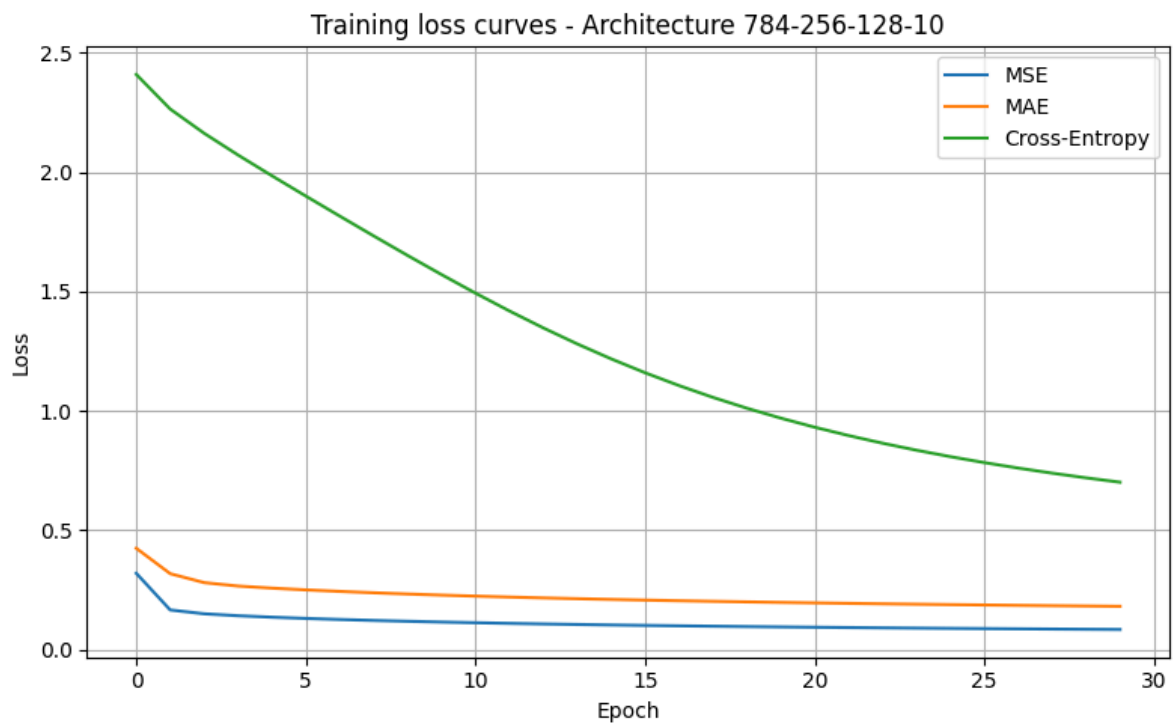


Figure 2 — Training loss curves for architecture 784-256-128-10



The Cross-Entropy loss starts around 2.3, which is expected for a random classifier with 10 classes because:

$$\log(0.1) \approx 2.302$$

The loss then decreases steadily during training, showing that the network is learning.

MSE and MAE have smaller numerical loss values, but these values are not directly comparable with Cross-Entropy because the losses use different mathematical scales. Therefore, final test accuracy is the main metric used to compare model quality.

Discussion

The results clearly show that Cross-Entropy is the most suitable loss function for MNIST multi-class classification. It achieved the highest accuracy for both tested architectures.

With the first architecture, Cross-Entropy reached an average test accuracy of 81.07%, while MSE reached only 46.72% and MAE reached 32.57%. With the second architecture, Cross-Entropy improved to 83.65%, while MSE and MAE remained much lower.

This difference is expected because Cross-Entropy is designed for classification problems. It works with probabilities produced by softmax and directly penalizes the model when it assigns low probability to the correct class. In contrast, MSE and MAE treat the task more like a regression problem on one-hot vectors. They do not directly optimize the probability distribution over classes.

MAE performed the worst. This can be explained by its gradient: $\text{sign}(y_p - y_t)$

The gradient only depends on the direction of the error, not on its magnitude. Therefore, MAE provides less informative updates during training. This makes convergence slower and less effective for this classification task.

MSE performed better than MAE, but still significantly worse than Cross-Entropy. MSE gives larger gradients for larger errors, which makes it more informative than MAE. However, it is still not ideal for multi-class classification because it does not directly optimize class probabilities.

The second architecture improved performance slightly compared to the first one. This improvement was most visible for Cross-Entropy, where the accuracy increased from 81.07% to 83.65%. This suggests that the deeper architecture can learn more complex representations, but the benefit is only useful when the loss function provides good training signals.

Conclusion

In this assignment, three loss functions were implemented and compared for MNIST classification using a NumPy-based Multilayer Perceptron.

The experiments show that Cross-Entropy is clearly the best loss function for this task. It achieved the highest accuracy, the most stable performance across seeds, and consistent convergence during training.

MSE was less effective because it treats classification as a regression problem on one-hot encoded vectors. MAE performed worst because its gradient only depends on the sign of the error and does not provide information about the magnitude of the mistake.

The best configuration was: $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$

with ReLU activations in the hidden layers and Cross-Entropy loss. This configuration achieved: $83.65\% \pm 0.91\%$ test accuracy.

Overall, the results confirm that Cross-Entropy is the most appropriate loss function for multi-class classification tasks such as MNIST.